RUNGTA INTERNATIONAL SKILLS UNIVERSITY
SCHOOL OF INFORMATION TECHNOLOGY

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>56</td></tr>
<tr><td>Project Title</td><td>Snake Game</td></tr>
<tr><td>Problem Statement</td><td>Create the classic Snake game with Pygame in which the player steers a snake to eat food, the snake grows and the score increases, and the game ends when the snake hits the wall or itself.
Language: Python 3 Concepts: Loops, Lists, Coordinates, Pygame
Note: Grid is 10 × 10 (x and y from 0 to 9). The snake starts as one segment at (5, 5) moving Right; U decreases y. Eating food adds 10 to the score and 1 to the length. The game ends on a wall or self collision. For testing, the first food is fixed at (7, 5), the next at (0, 0), and the moves are given as keys R, L, U, D.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Moves: R, R</td><td>Head: (6, 5)
Food eaten at (7, 5)!
Score: 10 | Length: 2 | Head: (7, 5)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Moves: R, R, R, R, R</td><td>Food eaten at (7, 5)! Score: 10
Head: (8, 5), then (9, 5)
Game Over! Wall hit at (10, 5).
Final score: 10</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Key: X
(b) Key: L (snake is moving Right)</td><td>(a) Invalid key. Use R, L, U or D.
(b) Reverse move not allowed. Snake continues Right.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Moves: R, R, U, U</td><td>Food eaten at (7, 5)! Score: 10
Snake: (7, 3), (7, 4)
Length: 2 | Head: (7, 3)</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console. Use the pygame library for the game window.
2. Store the snake as a list of coordinates and move it one cell at a time in the game loop.
3. Generate the food randomly, grow the snake and increase the score when the food is eaten.
4. Detect wall collision and self collision to end the game and show Game Over with the final score.
5. Control the snake with the arrow keys and do not allow an immediate reverse move.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>